

NEURAL NETWORKS AND DEEP LEARNING C1

Rare Loop Function:

A function which is never negative, it is the combination of a straight line in 1st quadrant and the horizontal constant line $y=0$, for all negative values of the straight line.

Neuron:

Transforms the input into output.

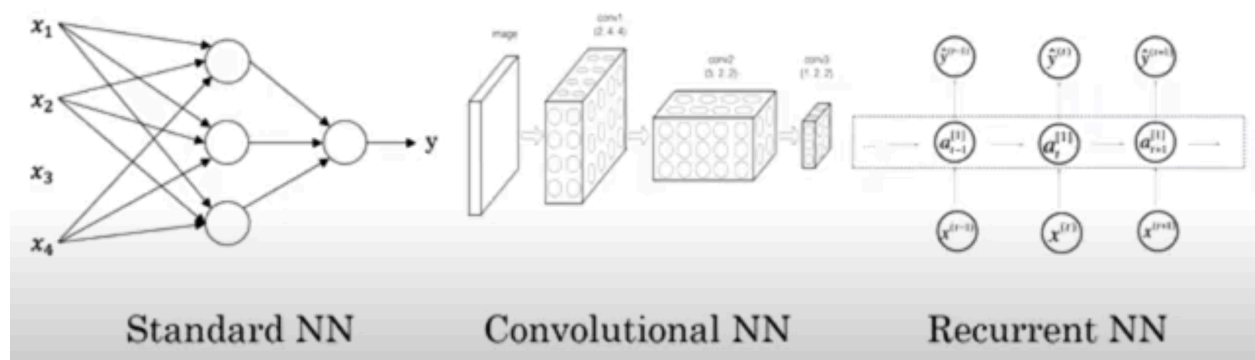
Weights:

During training, the network adjusts these weights to learn the relationships between inputs and outputs. Higher weights mean stronger influence of one neuron on another.

Biases:

Biases are added to the weighted sum of inputs within a neuron. They allow a neuron to activate even if all its inputs are zero.

Think of bias as a threshold that a neuron needs to overcome to "fire".



Supervised learning:

It essentially involves training a model on a set of input data that is paired with the correct output, allowing the algorithm to learn the relationship between the two.

Audio: RNN

Structured Data			Unstructured Data	
Size	#bedrooms	...	Price (1000\$)	
2104	3			
1600	3			
2400	3			
...	...			
3000	4			



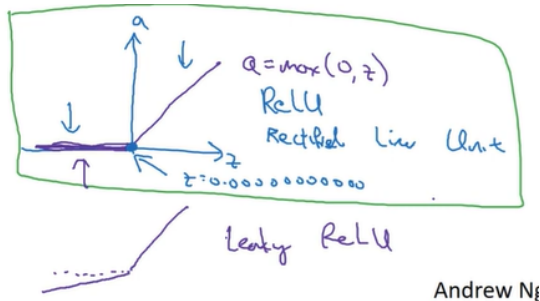

Audio

Image

Why sigmoid has been replaced with rectified linear unit (relu)?

Sigmoid functions has gradient almost zero at some places = learning almost zero.
Gradient descent works better on Relu.

A sigmoid function is a Activation Function.



Andrew Ng

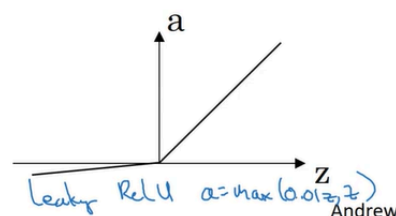
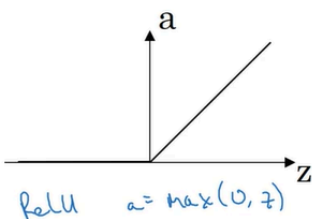
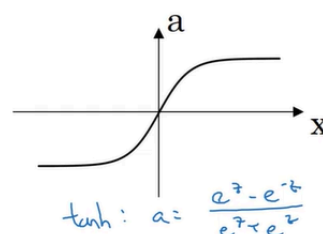
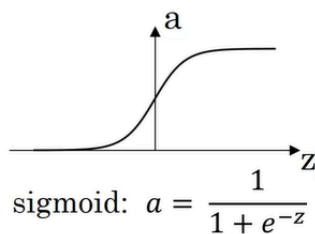
Sigmoid: Almost never use this, only use in output layer if you want 0 or 1 output.

Tanh: It is okay, but it has its derivative almost 0 at two places, so learning becomes slower.

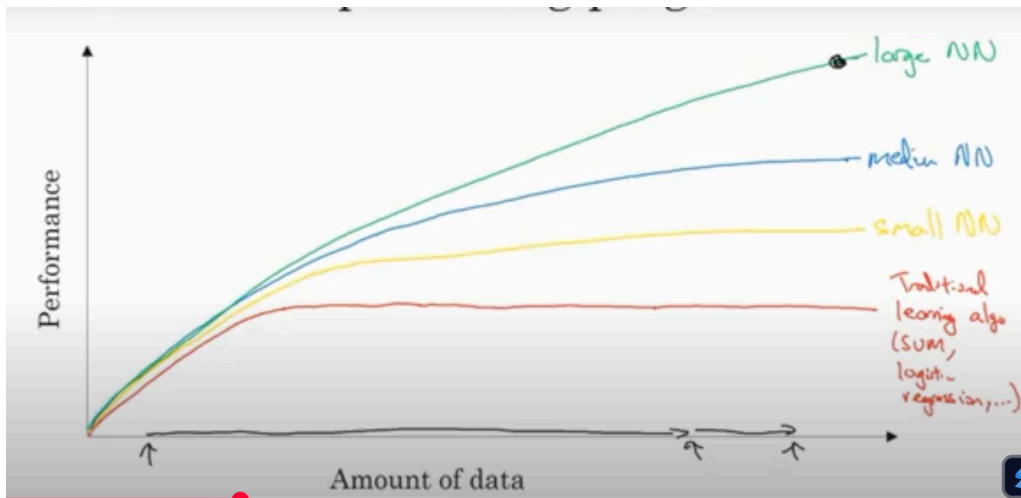
Relu: Again slope is zero for - ve values, but rest learning is very good and fast.

Leaky Relu: No zero slope.

Pros and cons of activation functions



Andrew



More data, bigger NN, more computation power.

Computational Process of NN:

Forward pass OR Forward propagation

Backward pass OR Back propagation

Binary Classification :

X feature vector = extracts features from input in form of a matrix.

Converts X feature vector into a binary output Y (0/1)

Logistic Regression :

Used in Supervised learning for binary classification problems.

Dim of w is the same as the input.

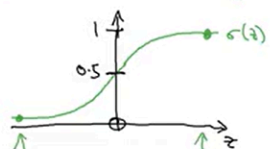
Dim of b is the same as the output.

Logistic Regression

Given x , want $\hat{y} = P(y=1|x)$
 $x \in \mathbb{R}^{n_x}$ $0 \leq \hat{y} \leq 1$

Parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$.

Output $\hat{y} = \sigma(\frac{w^T x + b}{z})$



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

If z large $\sigma(z) \approx \frac{1}{1+0} = 1$

If z large negative number

$$\sigma(z) = \frac{1}{1+e^{-z}} \approx \frac{1}{1+\text{Big num}} \approx 0$$

Andrew Ng

Loss Function : L

Used to find how good the output given by our model is compared to the expected output, calculated by Logistic Regression.

Applied on a single output pair.

Most common choice : $MSE = (y_{\text{model}} - y_{\text{expected}})^2 \times 0.5$

$$\underline{L(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2}$$

$y_{\text{model}} = \hat{y}$ = ground truth label

$\hat{y}$ is the probability of $y=1$ for input X .

But MSE does not deal with gradient descent properly.

$$\underline{L(\hat{y}, y) = - (y \log \hat{y} + (1-y) \log (1-\hat{y}))} \leftarrow$$

If $y=1$: $L(\hat{y}, y) = -\log \hat{y} \leftarrow$ Want $\log \hat{y}$ large, want $\hat{y}$ large.

If $y=0$: $L(\hat{y}, y) = -\log (1-\hat{y}) \leftarrow$ Want $\log (1-\hat{y})$ large ... Want $\hat{y}$ small



Above loss function is a convex function, local minima-the lowest loss or deviation from expected output for some w and b .

Cost Function : J

How well training is being done on the entire set.

Applied on entire set.

$$\begin{aligned} \rightarrow \text{ If } y = \underline{1}: & \quad p(y|x) = \hat{y} \\ \text{ If } y = \underline{0}: & \quad p(y|x) = 1 - \hat{y} \end{aligned} \quad \left. \vphantom{\begin{aligned} \rightarrow \text{ If } y = \underline{1}: & \quad p(y|x) = \hat{y} \\ \text{ If } y = \underline{0}: & \quad p(y|x) = 1 - \hat{y} \end{aligned}} \right\} p(y|x)$$
$$p(y|x) = \hat{y}^y (1-\hat{y})^{(1-y)} \leftarrow$$

$p(y|x)$ is the function for above 2 cases.

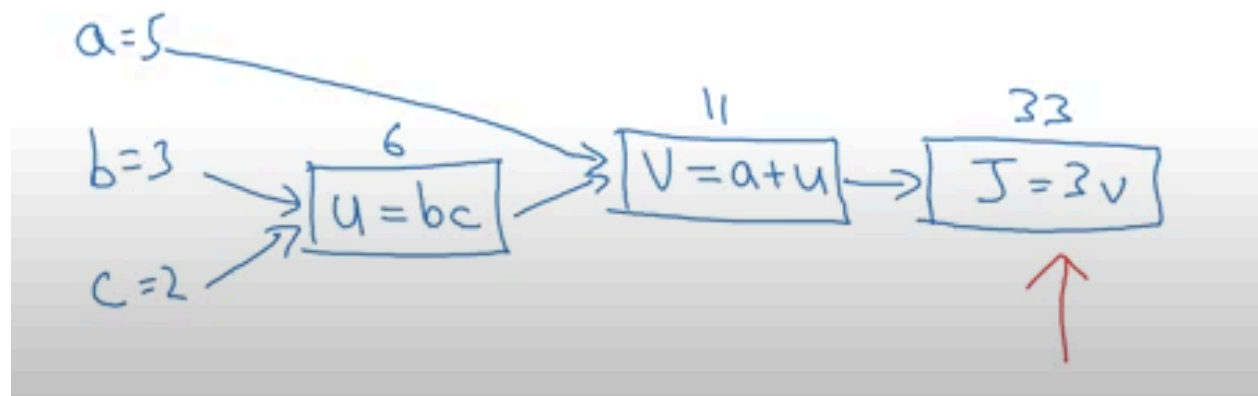
$$\begin{aligned}
 \log p(\text{labels in training set}) &= \log \prod_{i=1}^m p(y^{(i)} | x^{(i)}) \leftarrow \\
 \log p(\dots) &= \sum_{i=1}^m \underbrace{\log p(y^{(i)} | x^{(i)})}_{-\mathcal{L}(\hat{y}^{(i)}, y^{(i)})} \\
 &= -\sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\
 \text{Cost: } \underbrace{J(w, b)}_{\text{(minimize)}} &= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\
 J(w, b) &= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})]
 \end{aligned}$$

Maximum likelihood estimator $\nwarrow$

Computational graph:

J is cost fun that we need to minimize.

$$J = 3(a + bc)$$

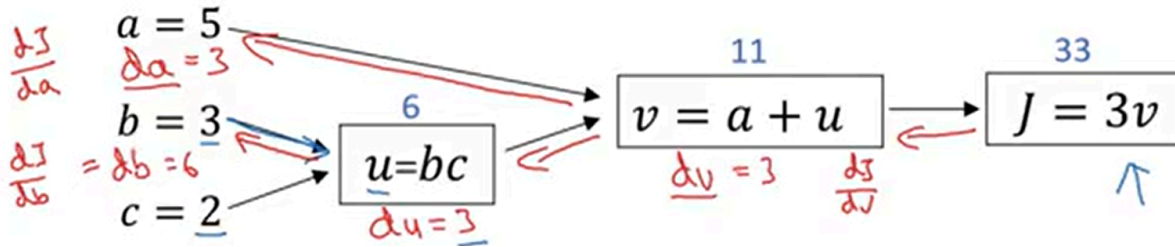


Derivatives:

Back propagation and Chain rule,

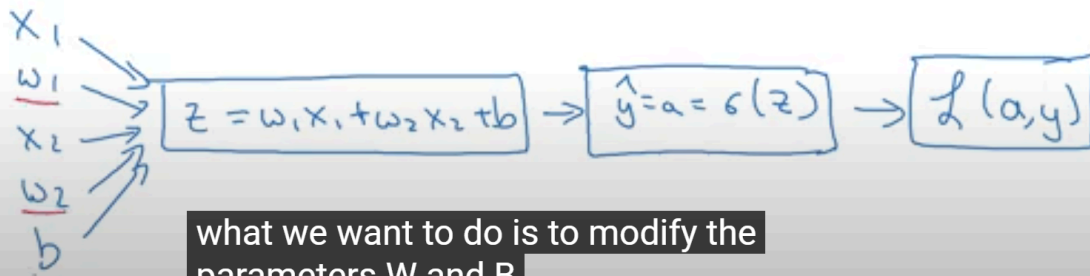
$$\frac{dJ}{dv} \quad \frac{dv}{da}$$

Computing derivatives



Logistic Regression Computation Graph:

- $\rightarrow z = w^T x + b$
- $\rightarrow \hat{y} = a = \sigma(z)$
- $\rightarrow \mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$



what we want to do is to modify the parameters W and B

Define:

$$u = 1 + e^{-x}$$

Rewriting the sigmoid function:

$$y = \frac{1}{u}$$

Differentiating u with respect to x :

$$\frac{du}{dx} = -e^{-x}$$

Differentiating y with respect to u :

$$\frac{dy}{du} = -\frac{1}{u^2}$$

Using the chain rule:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

$$\frac{dy}{dx} = \left(-\frac{1}{u^2}\right) \cdot (-e^{-x})$$

$$\frac{dy}{dx} = \frac{e^{-x}}{u^2}$$

Since $u = 1 + e^{-x}$, substituting:

$$\frac{dy}{dx} = \frac{e^{-x}}{(1+e^{-x})^2}$$

Since:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Rewriting:

$$1 - \sigma(x) = \frac{e^{-x}}{1+e^{-x}}$$

Substituting:

$$\frac{dy}{dx} = \sigma(x) \cdot (1 - \sigma(x))$$

Final Result

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

$$\frac{\partial}{\partial w_1} J(w, b) = \frac{1}{n} \sum_{i=1}^n \underbrace{\frac{\partial}{\partial w_1} \ell(a^{(i)}, y^{(i)})}_{\underline{dw_1^{(i)}} - (x^{(i)}, y^{(i)})}$$

Single step of gradient descent:

Logistic regression on m examples

$J=0$; $\underline{dw_1}=0$; $\underline{dw_2}=0$; $\underline{db}=0$
 For $i=1$ to m
 $z^{(i)} = w^T x^{(i)} + b$
 $a^{(i)} = \sigma(z^{(i)})$
 $J += -[y^{(i)} \log a^{(i)} + (1-y^{(i)}) \log(1-a^{(i)})]$
 $\underline{dz}^{(i)} = a^{(i)} - y^{(i)}$
 $dw_1 += x_1^{(i)} dz^{(i)}$
 $dw_2 += x_2^{(i)} dz^{(i)}$
 $db += dz^{(i)}$

$\uparrow \quad \uparrow \quad \uparrow$
 $n=2$

$J /= m$ ←
 $dw_1 /= m$; $dw_2 /= m$; $db /= m$. ←
 $\uparrow \quad \uparrow \quad \uparrow$

$\frac{\partial J}{\partial w_1}$
 $w_1 := w_1 - \alpha \underline{dw_1}$
 $w_2 := w_2 - \alpha \underline{dw_2}$
 $b := b - \alpha \underline{db}$

Andrew Ng

Do this m times.

Vectorization:

Non-vectorized:

```

z = 0
for i in range(n-x):
    z += w[i] * x[i]
z += b
        
```

Vectorized

$$z = \underbrace{w \cdot x}_{w^T x} + b$$

Vectorization is almost 300ms faster.

Avoid using explicit for loops for faster processing.

Numpy library has many Vectorized functions.

Broadcasting:

b is a real number, but to add it to a $1 \times m$ matrix python formats it into a $1 \times m$ matrix of b 's.

$$\underline{Z} = \begin{bmatrix} z^{(1)} & z^{(2)} & \dots & z^{(m)} \end{bmatrix} = \underbrace{w^T X}_{1 \times m} + \underbrace{[b \ b \dots b]}_{1 \times m} = \underline{Z}$$

$\rightarrow Z = \text{np.dot}(w.T, X) + \underline{b}$

Logistic Regression:

```
for iter in range(1000): ←  
    z = w.T X + b  
    = np.dot(w.T, X) + b  
    A = σ(z)  
    dz = A - Y  
    dw = 1/m X dz.T  
    db = 1/m np.sum(dz)  
  
    w := w - α dw  
    b := b - α db
```

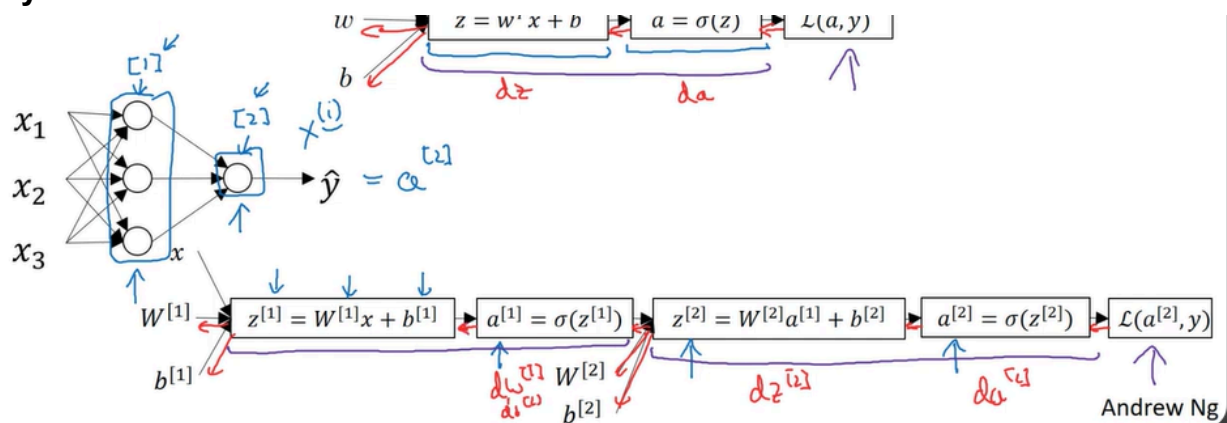
n't think

To sum column wise -> axis=0

To sum row wise -> axis=1

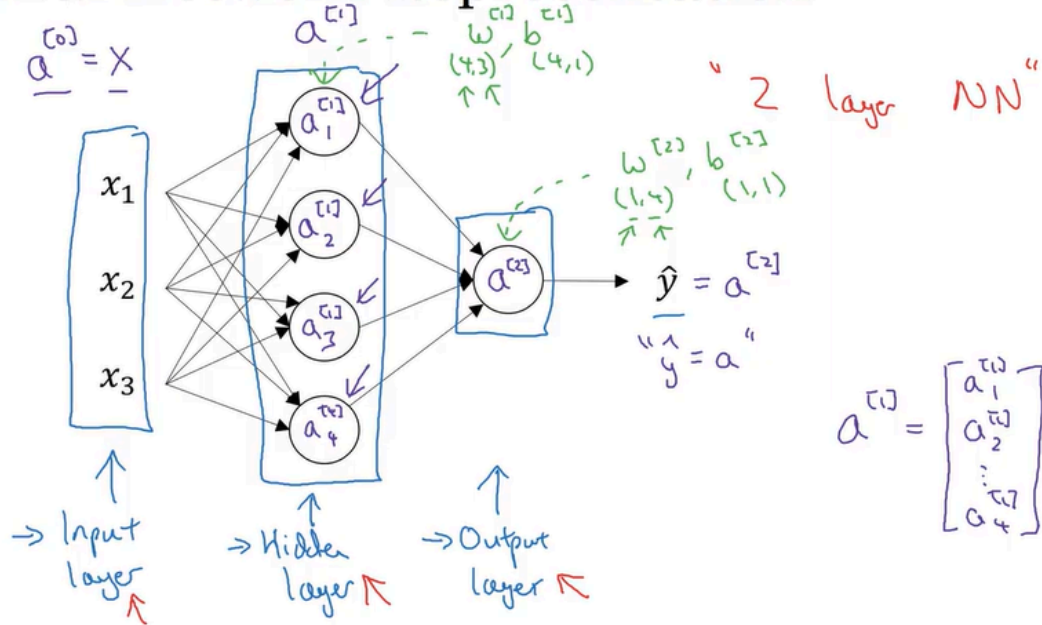
Don't use rank 1 arrays, use column or row vectors instead.

Layered NN:



Two Layer NN:

Neural Network Representation



$$z^{[1]} = W^{[1]}x + b^{[1]}, \quad a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}, \quad a^{[2]} = \sigma(z^{[2]})$$

Andrew Ng

Vectorization of a 2 layer NN:

Vectorizing across multiple examples

for $i = 1$ to m :

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

$$X = \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ \uparrow & & & \uparrow \\ & (n_x, m) & & \end{bmatrix}$$

$$z^{[1]} = W^{[1]}X + b^{[1]}$$

$$\rightarrow A^{[1]} = \sigma(z^{[1]})$$

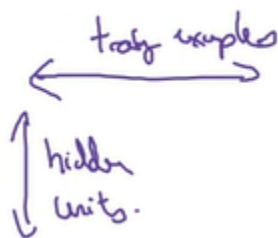
$$\rightarrow z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$\rightarrow A^{[2]} = \sigma(z^{[2]})$$

$$z^{[1]} = \begin{bmatrix} z^{1} & z^{[1](2)} & \dots & z^{[1](m)} \\ 1 & 1 & & 1 \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} a^{1} & a^{[1](2)} & \dots & a^{[1](m)} \\ 1 & 1 & & 1 \end{bmatrix}$$

Andrew Ng



The matrix $A[i]$ is

- Horizontally - the m training examples of the training set.
- Vertically - the individual nodes, like the left topmost refers to the 1st hidden layer ka 2st node.

Clearer explanation:

Justification for vectorized implementation

$$\begin{aligned} z^{1} &= W^{[1]}x^{(1)} + b^{[1]}, & z^{[1](2)} &= W^{[1]}x^{(2)} + b^{[1]}, & z^{[1](3)} &= W^{[1]}x^{(3)} + b^{[1]} \\ W^{[1]} &= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} & W^{[1]}x^{(1)} &= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} & W^{[1]}x^{(2)} &= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} & W^{[1]}x^{(3)} &= \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix} \\ W^{[1]} \begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} \end{bmatrix} &= \begin{bmatrix} \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \end{bmatrix} &= \begin{bmatrix} z^{1} & z^{[1](2)} & z^{[1](3)} \end{bmatrix} &= z^{[1]} \end{aligned}$$

Andrew Ng

What if $a[i] = z[i]$, a linear activation function?

It's actually useless, as the second layer ka computation becomes the same as the 1st layer but with different weights and biases, which we don't want. We want sequential processing.

$$\begin{aligned}
 a^{[1]} &= z^{[1]} = W^{[1]}x + b^{[1]} \\
 a^{[2]} &= z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \\
 a^{[2]} &= W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} \\
 &= \underbrace{(W^{[2]}W^{[1]})}_w x + \underbrace{(W^{[2]}b^{[1]} + b^{[2]})}_{b'} \\
 &= w'x + b'
 \end{aligned}$$

Full Computation for forward + back propagation:

Keepdims = true is for not making it a rank 1 array but a column vector.

Summary of gradient descent

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]}a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T}dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]}x^T$$

$$db^{[1]} = dz^{[1]}$$

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1, keepdims = True)$$

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g^{[1]'}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$$

$$J(\cdot) = \frac{1}{m} \sum_{i=1}^n \mathcal{L}(\hat{y}_i, y_i)$$

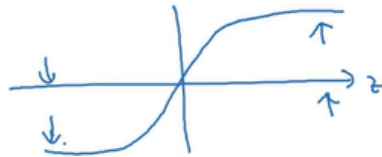
Random initialization:

If we initialize w with zeros, then the nodes in the same layer perform the same operations, this is not needed.

If the number that w is initialized to, is big then the tanh function will have slope zero, low learning rate.

Initialising b to zero is okay.

$$\begin{aligned} \rightarrow w^{[1]} &= \text{np.random.randn}(2,2) * \frac{0.01}{100?} \\ b^{[1]} &= \text{np.zeros}(2,1) \\ w^{[2]} &= \dots \\ b^{[2]} &= 0 \end{aligned}$$



$$\begin{aligned} z^{[1]} &= w^{[1]}x + b^{[1]} \\ a^{[1]} &= g^{[1]}(z^{[1]}) \end{aligned}$$

Generalised form for deep NN:

$$\begin{aligned} z^{[l]} &= w^{[l]} a^{[l-1]} + b^{[l]} \\ a^{[l]} &= g^{[l]}(z^{[l]}) \end{aligned}$$

$$dz^{[l]} = da^{[l]} * g^{[l]'}(z^{[l]})$$

$$dw^{[l]} = dz^{[l]} \cdot \underline{a^{[l-1]}}$$

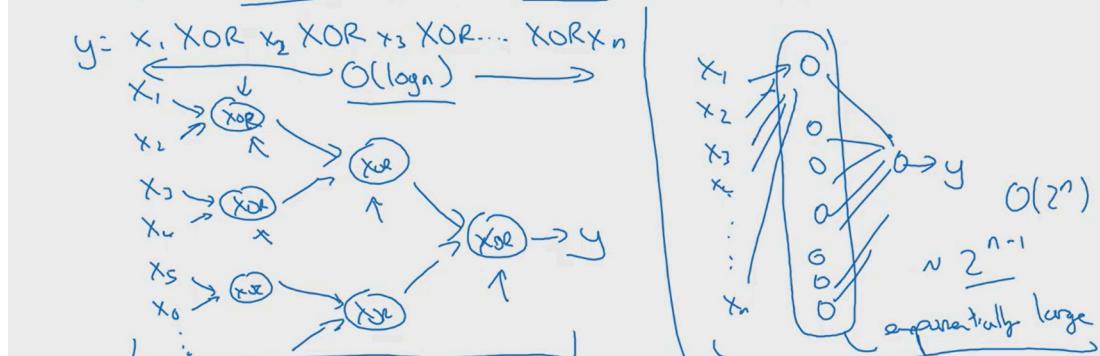
$$db^{[l]} = dz^{[l]}$$

$$da^{[l-1]} = w^{[l]T} \cdot dz^{[l]}$$

Circuit Theory:

Circuit theory and deep learning

Informally: There are functions you can compute with a “small” L-layer deep neural network that shallower networks require exponentially more hidden units to compute.



Summary:

Forward and backward functions

